Implémentation de données arborescentes

Implémentation récursive

De la même façon que les structures de données séquentielles, les données arborescentes sont des types abstraits de données qui peuvent être implémentée informatiquement de différentes façons.
Nous avons vu qu'il était possible de représenter un arbre binaire avec une listes dont la longueur dépendra de la hauteur de l'arbre. Cette représentation n'est pas très efficace car elle nécessite de coder de façon séquentielle une structure arborescente. Une solution plus efficace consiste à définir un arbre de façon récursive.

Mise en évidence de la récursion

En effet un arbre peut être vu comme un nœud comportant deux sous arbres gauche et droit (voir figure 1). Il s'agit donc d'une structure récursive puisque les sous arbres sont aussi des arbres
Le but de cet exercice est de définir une classe permettant d'implémenter un arbre binaire de façon récursive

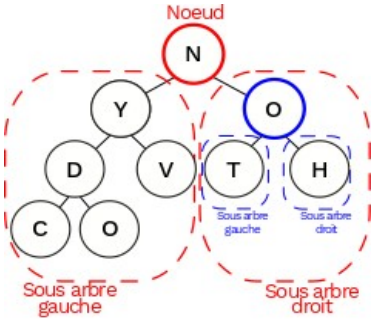


Figure 1: Représentation récursive d'un arbre

Expression minimale du TAD Arbre

Soit les fonctions de base du type abstrait de donnée Arbre sont les suivantes :

creer_arbre(d, gauche, droit) qui retourne un objet de type arbre.
La valeur de la racine est définie par d ; gauche et droit correspondent au contenu des sous arbres gauche et droit. Dans le cas d'une feuille, gauche et droit prennent la valeur None.
noeud(A) qui retourne la valeur du nœud de l'arbre A
sag(A) qui retourne le sous arbre gauche de l'arbre A
sad(A) qui retourne le sous arbre droit de l'arbre A
est_feuille(A) qui renvoie un booléen Vrai si le nœud est une feuille.

Exploitation des fonctions

On considère l'arbre défini sur la figure 2.

1. **Entourer** sur la figure 2 le sous arbre défini par l' instruction suivante :

```
creer_arbre('Y', creer_arbre('D', creer_arbre('C', None, None) , creer_arbre('O', None, None) ) , creer_arbre('V', None, None) )
```

2. **Compléter** cette instruction pour définir complètement l'arbre représenté sur la figure 2.

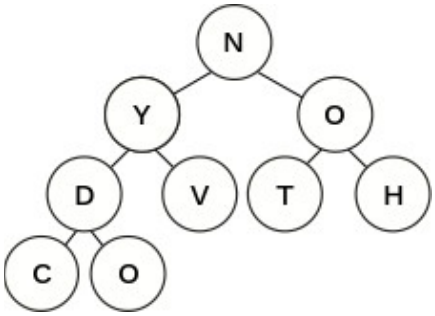


Figure 2: Arbre binaire étudié

Pour la suite cet arbre sera nommé abr.

Indiquer les valeurs renvoyées par les instructions suivantes :

```
noeud(sad(abr))
est_feuille(sag(sag(abr)))
est_feuille(sad(sad(abr))
```

3. **Indiquer** les instructions qui permettent d'effectuer les opération suivantes :
   ➔ Tester si le nœud T de abr est une feuille.
   ➔ Renvoyer la valeur du sous arbre droit du sous arbre gauche de l'arbre abr.

Programmation objet de l'implémentation récursive d'un arbre.

La programmation objet incomplète du TAD défini précédemment est la suivante :

```
class ArbreBin :
    def __init__(self,d, gauche=None, droit=None) :
    '''Construit un arbre constitue d'un noeud d'étiquette , d'un sous arbre gauche
    et un sous arbre droit.
    gauche = None et droit = None par defaut -> construction d'une feuille)'''
          self.__data = d
          self.__sag = gauche
          self.__sad = droit
    def est_feuille(self) :
    '''Retourne vrai si le noeud est une feuille'''
          return self.__sag==None and self.__sad==None
    def sag(self) :
          '''retourne le sous arbre gauche'''
          return self.__sag
    def sad(self) :
    '''retourne le sous arbre droit'''
          return self.__sad
    def noeud(self) :
    '''retourne l'etiquette du noeud'''
          return self.__data
```

Code disponible sur .\donnee\NSI\arbres\code\ArbreBin.py

Nombre de nœuds de l'arbre

On souhaite définir une méthode nbre_noeuds() qui renvoie le nombre de nœuds internes (sans les feuilles) d'un arbre binaire équilibré. Cette recherche peut se faire récursivement puisque ce nombre est égale au nombre de nœuds du sous arbre gauche plus le nombre du sous arbre droit plus un (lui même).

4. **Indiquer** la condition d'arrêt de cette fonction récursive nbre_noeuds()
5. **Programmer** la méthode nbre_noeuds() et **tester** son fonctionnement avec l'arbre de la figure 2 (variable abr)

Nombre de feuilles de l'arbre

On souhaite définir une méthode nbre_feuilles() qui renvoie le nombre de feuilles d'un arbre binaire équilibré.

6. **Programmer** la méthode nbre_feuilles() et **tester** son fonctionnement avec l'arbre de la figure 2 (variable abr)

Structures de l'arbre

7. **Programmer** une méthode hauteur() qui renvoie la hauteur de l'arbre. **Tester** son fonctionnement avec l'arbre de la figure 2 (variable abr)
8. **Programmer** une méthode feuilles() qui renvoie la liste des étiquettes des feuilles de l'arbre. **Tester** son fonctionnement avec l'arbre de la figure 2 (variable abr)